Loop Engineering is just K8s-style Cybernetics

From Watt's governor to Kubernetes to coding agents

Sayantan Das

MLOps North

November 5, 2026



1. The Steersman
2. Cybernetics in Ten Minutes
3. Kubernetes: Cybernetics for Infrastructure
4. Agents: Same Machine, New Actuator
5. loopctl: The Loop, Built
6. When Loops Go Wrong
7. A Small Brain That Decides
8. The Loop Above the Loop

κυβερνήτης
1788 · 1948 · 2014 · 2025

1 The Steersman

2 Cybernetics in Ten Minutes

3 Kubernetes: Cybernetics for Infrastructure

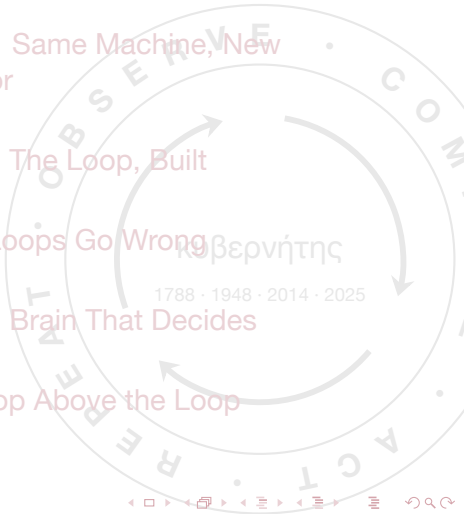
4 Agents: Same Machine, New Actuator

5 loopctl: The Loop, Built

6 When Loops Go Wrong

7 A Small Brain That Decides

8 The Loop Above the Loop



The title says the same thing twice

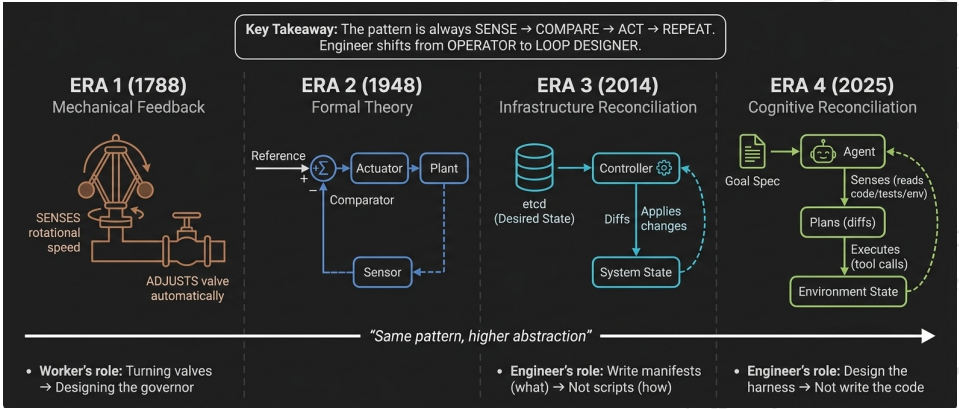
- κυβερνήτης (*kybernētēs*): the steersman of a ship
- **1948**: Norbert Wiener names the science of feedback and control cybernetics after it
- **2014**: Google open-sources a container orchestrator and names it Kubernetes: Greek for helmsman
- **2025**: we put a language model in the loop and call it loop engineering



Claim

This is not an analogy. It is the same machine at a higher level: **state a goal, sense, compare, act, repeat.**

Four eras, one pattern



Each era moves the human **up one level**: from turning the valve to designing the governor.

1 The Steersman

2 Cybernetics in Ten Minutes

3 Kubernetes: Cybernetics for Infrastructure

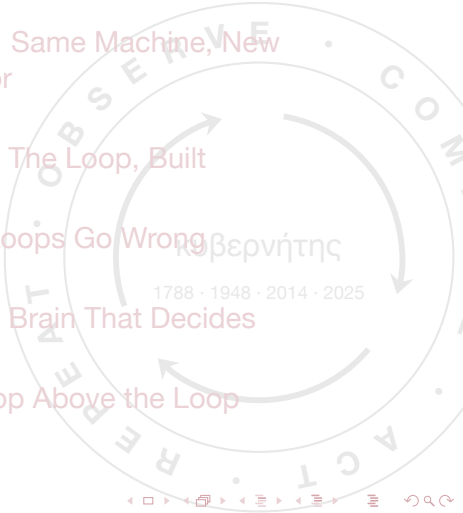
4 Agents: Same Machine, New Actuator

5 loopctl: The Loop, Built

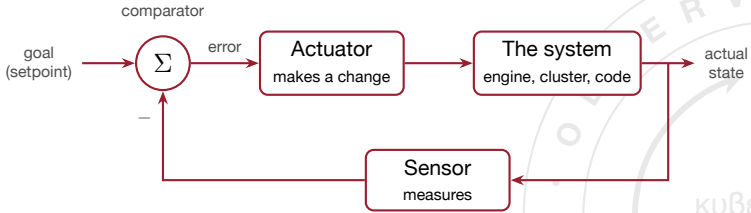
6 When Loops Go Wrong

7 A Small Brain That Decides

8 The Loop Above the Loop



Every feedback loop has the same four parts



	Goal	Sensor	Comparator	Actuator
Watt governor (1788)	target speed	spinning balls	linkage	steam valve
Thermostat	21 °C	thermometer	switch	furnace
Kubernetes (2014)	YAML manifest	API server watch	controller diff	controller writes
Agent loop (2025)	task spec + checks	tests, logs, browser	diff vs. goal	LLM edits code

How control loops fail: Maxwell, *On Governors* (1868)

Governors on steam engines hunted: they overshot, corrected, overshot again. Maxwell's paper on why is one of the founding texts of control theory.

Control term	What it means	In an agent loop
Negative feedback	push against the error	undo a change that made things worse
Damping	slow down after overshooting	smaller changes, wait longer after a miss
Oscillation (hunting)	overshoot, correct, repeat	fix one test, break two, fix those…
Steady-state error	settles short of the goal	stuck: the agent can't close the last gap
Saturation limits	the valve only opens so far	caps on rounds, time and money

Ashby's Law: you can only control what you can sense

Law of Requisite Variety (W. Ross Ashby, *An Introduction to Cybernetics*, 1956)

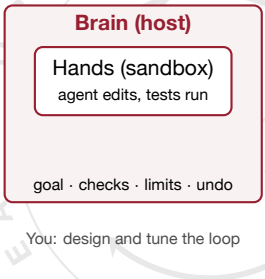
Only variety can absorb variety: a regulator can only correct the disturbances it can tell apart.

- For an agent loop the **checks are the sensors**. The loop can never be better than its checks.
- A bug the tests can't see is a bug the loop will happily ship.
- Goodhart's law is the dark side: a measure that becomes a target stops being a good measure.
- The cheapest way for an agent to make a failing test pass is to edit the test.

Invest in sensing before you invest in smarter planning.

Who steers the steersman?

- Second-order cybernetics: the observer is part of the system it observes.
- An agent that can touch its own sensors is grading its own homework.
- So split the loop:
  - **Brain outside**: goal, checks, limits, undo
  - **Hands inside**: edits and the code they run
- The human moves up again: you design and tune the loop, the loop does the work.



1 The Steersman

2 Cybernetics in Ten Minutes

3 Kubernetes: Cybernetics for Infrastructure

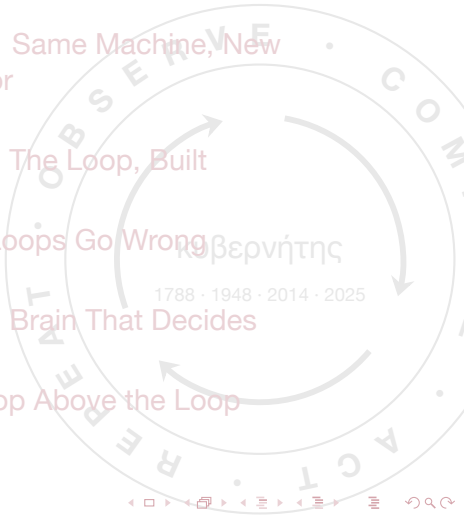
4 Agents: Same Machine, New Actuator

5 loopctl: The Loop, Built

6 When Loops Go Wrong

7 A Small Brain That Decides

8 The Loop Above the Loop



The reconciliation loop

1. **Desired state**: a manifest says what should exist
2. **Observe**: read the current state
3. **Diff**: compare desired vs. actual
4. **Act**: make the smallest change that closes the gap
5. **Repeat**: forever

Level-triggered, not edge-triggered

A controller reacts to *what is true now*, not to the event that just happened. Missed an event? The next pass still sees the gap.

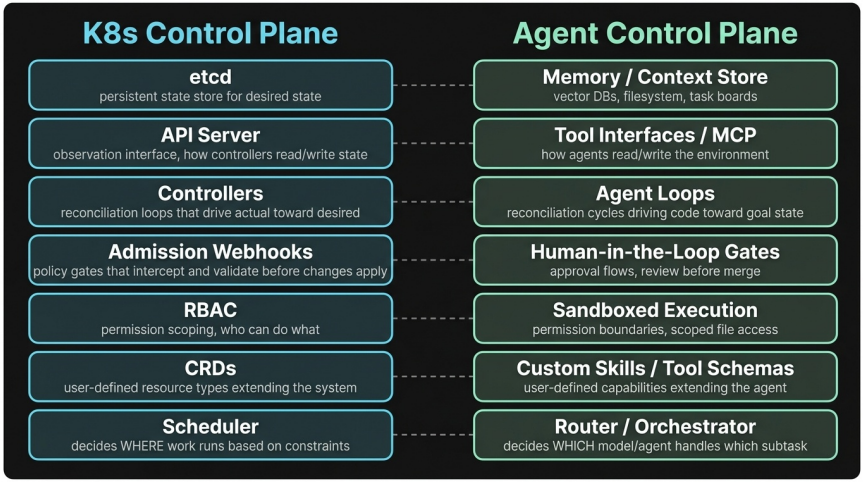
The controller never runs a script from A to B. It only knows how to make the *next correction*. Kill a pod and it comes back: self-healing falls out of the loop. Nobody writes a special case for it.

What a controller looks like (controller-runtime, sketch)

```go
func (r *ModelReconciler) Reconcile(ctx context.Context,
        req ctrl.Request) (ctrl.Result, error) {
    var m mlv1.Model
    if err := r.Get(ctx, req.NamespacedName, &m); err != nil { // observe
        return ctrl.Result{}, client.IgnoreNotFound(err)
    }
    if m.Status.ServedVersion == m.Spec.Version {     // diff
        return ctrl.Result{}, nil                      // converged
    }
    if err := r.rolloutOneStep(ctx, &m); err != nil { // act: one step
        return ctrl.Result{}, err                      // retried, backoff
    }
    return ctrl.Result{RequeueAfter: 30 * time.Second}, nil // repeat
}
```

A hypothetical `Model` resource. Note what's missing: no plan, no sequence of steps, no memory of the last run.

The control plane, mapped



MLOps already runs on these loops

- **GitOps** (Argo CD, Flux): git holds the desired state, a controller syncs the cluster to it.
- **Continuous training**: a metric drops below the line, retrain until it's back above.
- **Progressive delivery**: canary metrics go bad, roll back automatically.
- **Data validation before training**: a gate that rejects bad input, like an admission webhook.

So what's new?

Only the actuator. The thing making the change is now a language model, and it can make *any* change, including bad ones. Everything else you already know.

1 The Steersman

2 Cybernetics in Ten Minutes

3 Kubernetes: Cybernetics for Infrastructure

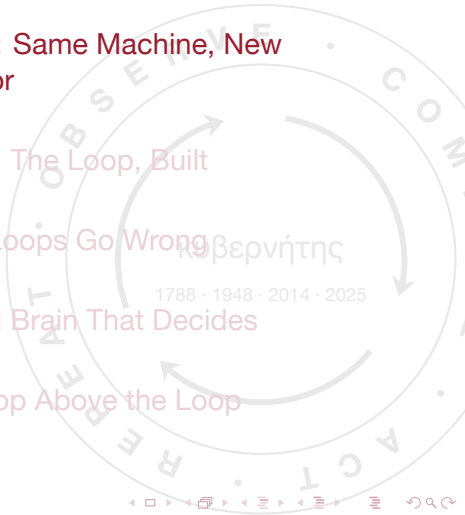
4 Agents: Same Machine, New Actuator

5 loopctl: The Loop, Built

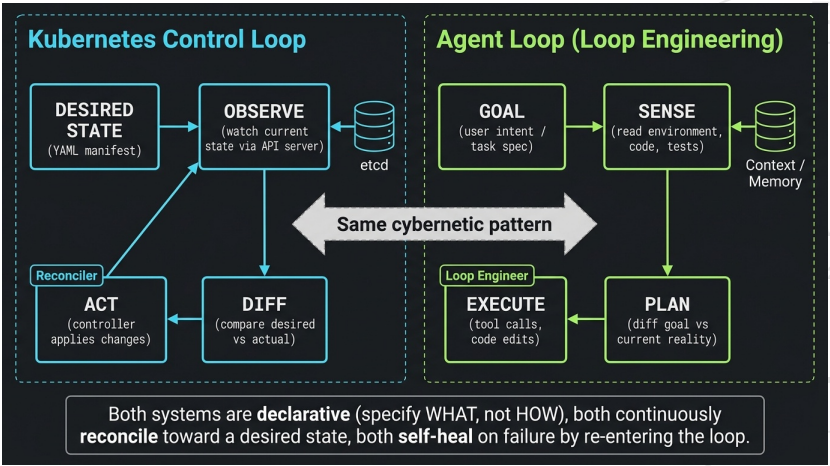
6 When Loops Go Wrong

7 A Small Brain That Decides

8 The Loop Above the Loop



Same loop, new actuator



Design rules that transfer directly

Rule	In Kubernetes	In an agent loop
Declarative	manifest says *what*, not *how*	“all checks pass” beats a 47-step checklist
Observe first	read the API server before acting	read the code and run the tests before editing
Small diffs	change one thing per pass	one target per round, cap the diff size
Idempotent	re-running a pass is harmless	“file contains X”, not “append Y”
Crash-only	controllers can die anytime	progress lives in git, not in the agent’s head
Bounded ownership	labels and selectors	one loop owns one directory

Idempotency bit me while writing the demo

The scripted demo worker fixes one bug per run by replacing text in `calc.py`:

```
# First version: replace the first `return a - b`
FIXES = {"add": ("return a - b", "return a + b"), ...}
# Run twice: once sub is fixed to `return a - b`, the add fix hits sub

# Fixed version: anchor on context, so a second run is a no-op
FIXES = {"add": ("def add(a, b):\n    return a - b",
                 "def add(a, b):\n    return a + b"), ...}
```

“Make the file *contain* X” converges. “Do Y to the file” drifts every time the loop re-runs it.

1 The Steersman

2 Cybernetics in Ten Minutes

3 Kubernetes: Cybernetics for Infrastructure

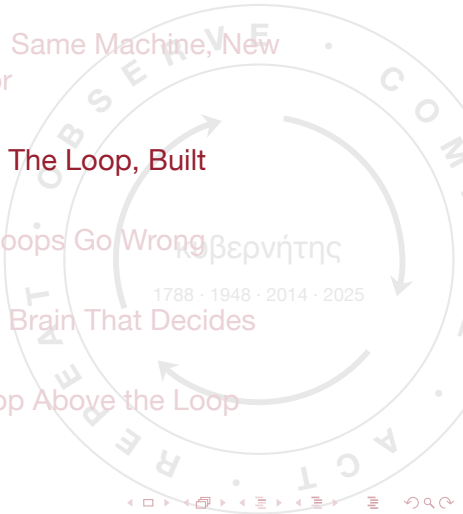
4 Agents: Same Machine, New Actuator

5 loopctl: The Loop, Built

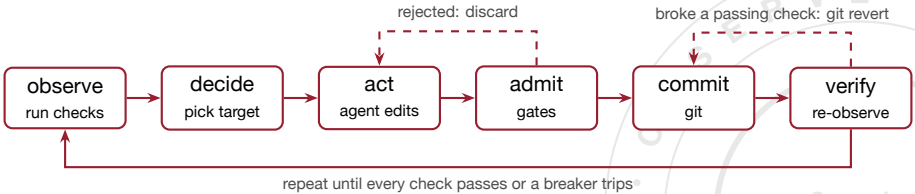
6 When Loops Go Wrong

7 A Small Brain That Decides

8 The Loop Above the Loop



loopctl in one picture



- About 970 lines of Python, standard library only. Git is its etcd: every kept change is a commit.
- The agent is just a shell command. pi, Claude Code, Codex, or a script.
- Kill it at any point; rerun picks up where it left off, with budgets intact.

The goal is a file

```toml
name  = "calc"
goal  = "calc.py arithmetic behaves as test_calc.py specifies."
actor = 'pi -p "$(cat)"'        # any agent CLI; the prompt arrives on stdin
fresh_clone = true              # observe a clean clone every round: no stale reads

[[checks]]                      # desired state = every check exits 0
name = "add"
run  = "python3 -m unittest -q test_calc.TestCalc.test_add"

[[admission]]                   # policy gate on every proposed change
name = "tests-unchanged"
run  = "git diff --cached --quiet -- test_calc.py"

[limits]                        # the safety governor
max_iterations = 10
max_diff_lines = 20             # damping: big changes are rejected
stall_iterations = 3            # steady-state error: give up
max_regressions = 2             # oscillation: stop thrashing
```

Every part has a cybernetic name

loopctl	Cybernetics	Kubernetes
`goal` + `[[checks]]`	setpoint	manifest
running the checks	sensor	watch via API server
passing vs. failing	comparator	diff
the agent (`actor`)	actuator	controller writes
`git revert` on regression	negative feedback	rollback
backoff, `max_diff_lines`	damping	rate-limited requeue
`max_regressions`	oscillation guard	—
`stall_iterations`	steady-state error	—
`[[admission]]`, approval	—	admission webhooks
lease file, sandbox	—	RBAC, namespaces
`[decider]`	controller logic	scheduler

Demo 1: the loop converges (offline, scripted worker)

```
$ ./examples/calc/demo.sh
[calc] observe: 1/4 passing, failing ['add', 'sub', 'div']
[calc] act #1: target `add`
[calc] committed 7f0e20ae (2 lines)
[calc] observe: 2/4 passing, failing ['sub', 'div']
[calc] act #2: target `sub`
[calc] committed c4470cd2 (2 lines)
[calc] observe: 3/4 passing, failing ['div']
[calc] act #3: target `div`
[calc] committed 7a07aa94 (2 lines)
[calc] observe: 4/4 passing
[calc] converged
```

One target per round, observe after every change, stop when the sensors say done. Runs in about 2 seconds with no network.

Demo 2: a real agent as the actuator

```
1 actor = 'pi -p "$(cat)"'
```

```
$ ./examples/calc/demo.sh loop.pi.toml
...
[calc-pi] observe: 4/4 passing
[calc-pi] converged
calc-pi: 3 actions, 3 commits,
  0 rejected, 0 reverted, 23.6s acting
```

pi's fix for `div` was not the scripted one:

```
--- loop(calc-pi): #3 div
-     if b == 0:
-         return 0
```

It deleted the guard so Python raises `ZeroDivisionError` itself. Different path, same goal. That's what "declare *what*, not *how*" buys you.

Demo 3: brain outside, hands inside

Each check runs in a throwaway Apple container VM:

```
container run --rm --network none \
  -v "$PWD":/work:ro -w /work \
  python:3.12-slim "$@"
```

```
1 run = "./incvm python3 -m unittest ..."
```

What agent-written code sees inside:

```
home visible: False
network: blocked URLError
write /work: blocked Read-only file system
```

loopctl and the agent stay on the host, so the agent can't rewrite how it's graded. About 1 s per VM; the full loop converged in 55 s.

1 The Steersman

2 Cybernetics in Ten Minutes

3 Kubernetes: Cybernetics for Infrastructure

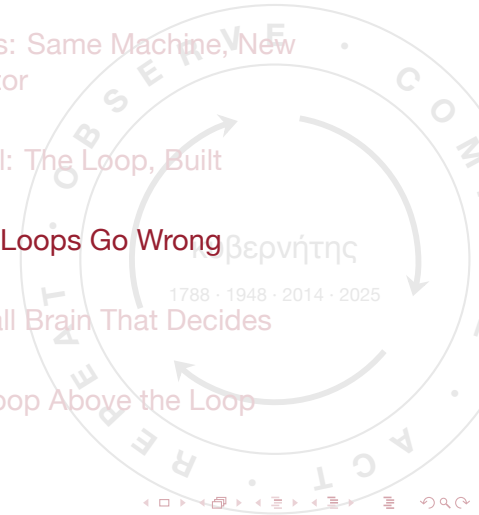
4 Agents: Same Machine, New Actuator

5 loopctl: The Loop, Built

6 When Loops Go Wrong

7 A Small Brain That Decides

8 The Loop Above the Loop



Kubernetes already learned these lessons

Failure	Looks like	loopctl's answer
Thrashing	fix one test, break two	verify every commit; revert on regression; stop after `max_regressions`
Stale reads	acting on an old picture	re-run checks every round, optionally on a fresh clone
Endless loop	goal can't be reached	caps on rounds and time; `stall_iterations`
Fighting loops	two loops, one directory	lease file; edits outside the directory halt the loop
Gaming the sensor	agent edits the test	admission checks reject the change
Blind loop	no signal to steer by	nothing to fix in the loop: add checks

Live failure: thrashing gets caught

A worker that fixes `add` but breaks `mul` every time:

```
[calc] observe: 1/4 passing, failing ['add', 'sub', 'div']
[calc] act #1: target `add`
[calc] committed 84dcb427 (4 lines)
[calc] observe: 1/4 passing, failing ['sub', 'mul', 'div']
[calc] reverted 84dcb427: broke ['mul']
[calc] act #2: target `add`
[calc] committed 8a047e50 (4 lines)
[calc] observe: 1/4 passing, failing ['sub', 'mul', 'div']
[calc] reverted 8a047e50: broke ['mul']
[calc] thrashing: 2 reverted regressions
```

Negative feedback (revert), then a hard stop instead of hunting forever. Every attempt stays in git history for a human to read.

Live failure: the agent tries to move the goalposts

A worker that “fixes” `add` by changing the expected answer in the test:

```
[calc] act #1: target `add`
[calc] rejected: admission[tests-unchanged]: exit 1
...
[calc] unreachable: no progress in 3 iterations; still failing ['add', 'sub', 'div']
```

- Goodhart’s law, blocked by an admission webhook: one line of shell.
- The loop gives up instead of shipping a fake green build.
- `loopctl report` shows the pattern: 3 rejections, all `admission[tests-unchanged]`.

1 The Steersman

2 Cybernetics in Ten Minutes

3 Kubernetes: Cybernetics for Infrastructure

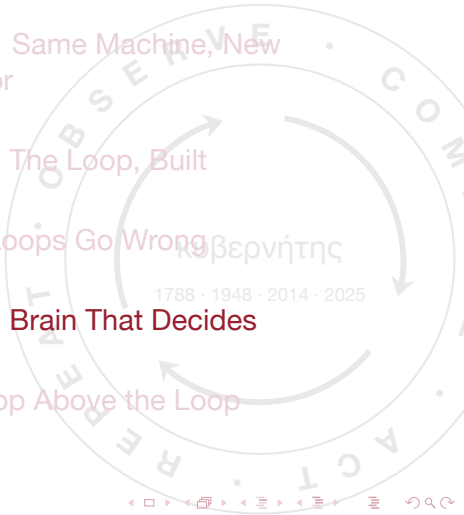
4 Agents: Same Machine, New Actuator

5 loopctl: The Loop, Built

6 When Loops Go Wrong

7 A Small Brain That Decides

8 The Loop Above the Loop



The scheduler doesn't run pods. It picks.

Kubernetes scheduler

- filter: which nodes *can* run this pod?
- score: which is best?
- pick one; the kubelet does the work

Same split for agents

- a small, fast model **decides**
- a big, slow model **does**
- the checks **judge**

TypeSafe Jev

Answers typed questions with probabilities instead of free text. One request per round:

- move: act / ask a human / stop
- target: which failing check next

What the decider is not allowed to do

- **Judge.** It can't mark anything as passed. Only the checks converge the loop.
- **Pick outside the menu.** A passing check as target is rejected; that round uses the built-in rule.
- **Act on a shaky call.** Under 50% confidence, or "ask a human": loopctl stops and asks.
- **Override the governor.** Caps on rounds, time, thrashing and stalling still apply.
- **Break the loop.** If it crashes or times out, the loop falls back to the simple rule.

Typed output guarantees the *shape* of the answer, not that it's right. Keep the judge outside.

Rules vs. Jev: an impossible check listed first

`gpu` runs `nvidia-smi`: no code change can make it pass on a Mac.

Built-in rule (first failing check)

```
act #1: target `gpu`
rejected: no_op: actor made no changes
act #2: target `gpu`
rejected: no_op: actor made no changes
act #3: target `gpu`
rejected: no_op: actor made no changes
unreachable: no progress in 3 iterations
```

0 of 3 fixable checks fixed.

Jev decider

```
decide: act `add` (0.98)
  target add=0.67 div=0.22 sub=0.09 gpu=0.02
committed 22ef9645
decide: act `sub` (0.98)
  target sub=0.90 div=0.10 gpu=0.00
committed 6f7b3f5c
decide: act `div` (0.96)
  target div=1.00 gpu=0.00
committed 3d5f5a34
decide: stop `gpu` (0.97)
  stop=0.99 ask_human=0.01 act=0.00
unreachable: decider chose stop
```

3 of 3 fixed, then stopped by itself. Whole run: about 3 s.

1 The Steersman

2 Cybernetics in Ten Minutes

3 Kubernetes: Cybernetics for Infrastructure

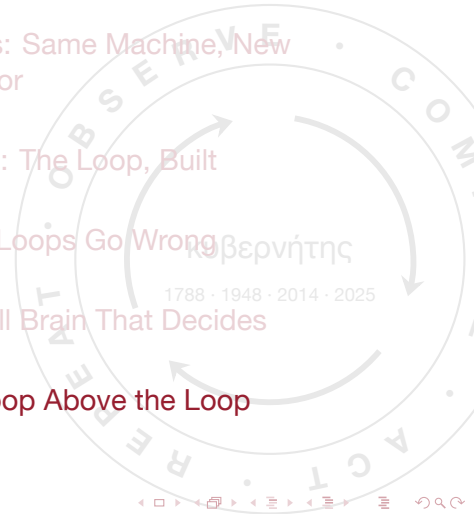
4 Agents: Same Machine, New Actuator

5 loopctl: The Loop, Built

6 When Loops Go Wrong

7 A Small Brain That Decides

8 The Loop Above the Loop



The hill-climbing machine

The same loop, one level up:

1. ship a v0 agent
2. record what it does (traces)
3. mine traces for failure patterns
4. turn failures into checks
5. improve the agent; repeat

The setpoint is now *agent quality*. You are the actuator.

```
$ loopctl report loop.jev.toml
calc-jev: 3 actions, 3 commits, 0 rejected,
  0 reverted, 0.1s acting
last outcome: unreachable (decider chose stop)

checks observed failing:
     4  gpu
     3  div
     2  sub
     1  add

decisions:
     3  decider: act
     1  decider: stop
```

Your turn (5 min): write the loop for a model

```
name  = "churn-model"
goal  = "The churn model is fit to ship."
actor = 'pi -p "$(cat)"'

[[checks]]
name = "schema"      # data contract holds
run  = "python validate.py data/train.parquet"
[[checks]]
name = "quality"     # setpoint on a metric
run  = "python eval.py --min-auc 0.85"

[[admission]]
name = "holdout-untouched"
run  = "git diff --cached --quiet -- eval.py
    data/holdout/"
```

With your neighbour:

1. If *you* were the agent, how would you cheat these checks?
2. What could go wrong that no check here can see? (Ashby)
3. When should this loop stop and ask a human?
4. Which limit would you set first?

Takeaways

1. **Declare the exit condition, not the path.** Checks are your setpoint.
2. **Your checks are your sensors.** The loop is only as good as what it can see.
3. **Never let the actuator touch the sensor.** Brain outside, hands inside.
4. **Observe before every action.** Don't trust the agent's memory.
5. **One small change per round, verified.** Undo anything that makes it worse.
6. **Keep actions safe to repeat, and state outside the agent.** Then you can kill it anytime.
7. **Install a governor.** Caps on rounds, time and money, plus a human gate.
8. **Let a small model decide and a big model do.** The checks still judge.

This isn't new. It's Kubernetes for thinking, and Watt's governor for software.

Further reading

- S. Das, *Loop Engineering Is Just K8s-Style Orchestration* (blog.ucalyptus.me)
- N. Wiener, *Cybernetics: Or Control and Communication in the Animal and the Machine*, 1948
- W. R. Ashby, *An Introduction to Cybernetics*, 1956
- J. C. Maxwell, *On Governors*, Proceedings of the Royal Society, 1868
- controller-runtime and the Kubernetes scheduler docs
- Apple container and pi-cvm (github.com/ucalyptus/pi-cvm)
- TypeSafe docs (docs.typesafe.ai)

OBSERVE · COMPREHEND · ACT · REPEAT
κυβερνήτης
1788 · 1948 · 2014 · 2025

Thanks!


OBSERVE · COMPREHEND · ACT · REPEAT
κυβερνήτης
1788 · 1948 · 2014 · 2025